REG / REG+ Specification Guide

Version: 1.0 · **Date:** 2026-10-19 **Scope:** Normative specification guide for REG and REG+, with formal property definitions, interface specification, evidence model, and three concrete use cases.

Table of Contents

1. What REG Is
2. What REG+ Adds
3. The Four Consensus Properties
4. The ConsensusBackend Interface
5. The Parametric Fault Model
6. Evidence and Conformance
7. Use Case 1: Financial Mandate Enforcement
8. Use Case 2: Cloud Infrastructure Budget Governance
9. Use Case 3: Multi-Agent Procurement Coordination

---

1. What REG Is

REG (Runtime Execution Governance) is an implementation-neutral standard that defines **what properties** a runtime governance system must guarantee when AI agents take consequential actions — not **how** to implement them.

The core insight: existing governance frameworks (ISO/IEC 42001, NIST AI RMF, EU AI Act) define management-system requirements and risk taxonomies, but none produces a **verifiable runtime artifact** that proves governance was enforced at the moment of action. REG fills this gap by specifying:

- **Properties** the runtime must guarantee (safety, liveness, fault tolerance, budget conservation)
- **An interface** any backend can implement to claim conformance

- **Evidence types** that attest whether claimed capabilities are actually demonstrated
- **A fail-closed verifier** that rejects unattested claims

REG is deliberately **not** an implementation. It is a contract. TENIR-Gov is one implementation; others could conform to the same contract with different mechanisms.

Design Principle: Properties, Not Mechanisms

> The standard defines the properties, not the mechanism.

A backend of consensus is conforming to this contract if it satisfies the properties C1, C2, C3, and I8 for the parameters declared in the profile. The standard does not prescribe Raft, Paxos, etcd, or PostgreSQL. It prescribes what the backend must guarantee.

The interface `ConsensusBackend` (see `consensus-interface-v1.json`) is the point of conformance. Any backend that implements this interface and satisfies the properties is conforming.

---

2. What REG+ Adds

REG+ extends the base REG contract with **consensus and budget coordination** — the layer that handles concurrent, cross-process, and cross-host resource governance:

Layer	REG	REG+
Decision	Adjudication, policy evaluation, geometry	Same
Execution boundary	`execution_allowed` gate on side effects	Same
Budget	Per-operation reservation	**Aggregate budget invariant (I8)** across concurrent operations
Consensus	Not specified	**ConsensusBackend interface** with C1/C2/C3 properties

Layer	REG	REG+
Fault model	Not specified	**Parametric fault model** (crash, byzantine, omission)
Evidence	Decision receipts	**Structured evidence logs** with SHA-256 attestation

REG+ does not replace REG — it adds the coordination layer that REG left unspecified. A system can be REG-conformant without REG+ (single-operation governance), but REG+ is required for multi-operation, multi-agent, or distributed governance.

---

3. The Four Consensus Properties

The consensus contract (`consensus-contract-v1.md`) defines four properties that any conforming backend must satisfy:

C1 — Safety / Linearizability

Every operation (`reserve` , `commit` , `release` , `reconcile`) takes effect at a single point between its invocation and response. The order of operations is a linearizable order that respects real-time precedence of non-overlapping operations.

Formally: for any execution history `H` , there exists a serialization `S` such that:

1. `S` is equivalent to `H` (same operations, same results)
2. `S` respects the precedence relation of `H` (if op_1 terminates before op_2 begins, then op_1 precedes op_2 in `S`)
3. `S` satisfies the sequential specification of each operation

Verification: Wing-Gong linearizability checker or equivalent.

C2 — Liveness / Termination

Under a healthy quorum (at least `quorum` nodes alive and communicating), every non-blocked operation eventually terminates. There exists a bounded time `T_max` such that any operation invoked at t_0 terminates before t_0 + `T_max` , provided the quorum is maintained during $[t_0, t_0 + T_max]$.

Liveness is **conditional**: under partition or crash exceeding `f`, the system may block — that is acceptable. What is **not** acceptable is violating safety under partition.

C3 — Fault Tolerance

The system maintains C1 (safety) and I8 (budget) under the declared fault model:

- `f` nodes crash (crash-stop) at arbitrary times
- `m` nodes produce arbitrary responses (byzantine)
- `o` nodes lose messages (omission)

Safety must hold **during** the fault, not only after recovery.

Recovery: A node that crashes and restarts must:

1. Reload its state from durable storage
2. Catch up on missing operations via the replicated log
3. Not resume operations already committed by other nodes

I8 — Aggregate Budget Conservation

At every instant, for each resource:

```
Σ(reserved) + Σ(committed) ≤ capacity
```

Where:

- `Σ(reserved)` = sum of amounts reserved but not yet committed or released
- `Σ(committed)` = sum of amounts committed but not yet released
- `capacity` = the declared budget ceiling

This must hold at **every intermediate state**, not just the final state. Under concurrency, two simultaneous `reserve()` calls must not cause the total to exceed capacity even if each individually sees sufficient available budget.

Scope: I8 applies per `resource_id`. The profile may declare multiple resources, each with its own ceiling. The invariant is per-resource, not global.

---

4. The ConsensusBackend Interface

The interface (`consensus-interface-v1.json`) defines five operations:

```
reserve(key, amount, resource_id) → ReservationResult
commit(reservation_id) → CommitResult
release(reservation_id) → ReleaseResult
reconcile(key) → ReconciliationResult
get_state(resource_id) → ConsensusState
```

Operation Specifications

`reserve(key, amount, resource_id)`

- Precondition: `amount > 0`
- Postcondition: if success, `reservation_id` unique created, `reserved += amount`, `available -= amount`
- Failure: if `available < amount`, returns `INSUFFICIENT_BUDGET` without effect
- Idempotence: `reserve(key, amount)` with an existing key returns the existing reservation if the amount matches, otherwise `DUPLICATE_KEY`

`commit(reservation_id)`

- Precondition: `reservation_id` exists and is not committed
- Postcondition: `reserved -= amount`, `committed += amount`
- Failure: if not found → `NOT_FOUND`; if already committed → `ALREADY_COMMITTED`; no effect in either case

`release(reservation_id)`

- Precondition: `reservation_id` exists and is not committed
- Postcondition: `reserved -= amount`, `available += amount`
- Failure: if not found → `NOT_FOUND`; if already committed → `ALREADY_COMMITTED`; no effect

`reconcile(key)`

- Precondition: `key` has an operation in `UNKNOWN` state

- Postcondition: returns `CONFIRMED` (effect occurred), `FAILED` (no effect), or `UNRESOLVED` (provider unreachable)
- If `CONFIRMED`: `committed += amount` if not already committed
- If `FAILED`: `reserved -= amount`, `available += amount`

`get_state(resource_id)`

- Postcondition: returns `{ available, reserved, committed, capacity, reservations: [...] }`
- Pure: does not modify any state

Key Design Decisions

- **`key` is an idempotency key** — prevents double-reservation of the same logical operation
- **`reservation_id` is generated client-side** (in the Raft implementation) to ensure deterministic replication — generating UUIDs inside a `@replicated` method violated Raft's determinism requirement
- **`reconcile()` handles UNKNOWN states** — when a leader crashes mid-operation, the reservation may be indeterminate; reconciliation queries the provider to determine whether the effect occurred
- **`get_state()` is pure** — on the leader, linearizable; on followers, eventually consistent

The interface is **implementation-neutral**: a file-lock backend, a Raft cluster, a PostgreSQL transaction, or an etcd-backed coordinator can all conform, provided they satisfy C1/C2/C3/I8 for their declared fault model.

Invariants

The interface defines four invariants that the backend must maintain at all times:

Invariant	Property
I8	`reserved + committed ≤ capacity` (per `resource_id`)
Linearizability	A linearizable history exists (verified by Wing-Gong or equivalent)
Agreement	At most one commit per reservation

Invariant	Property
Validity	No phantom operations (every reservation/commit/release corresponds to a prior valid operation)

---

5. The Parametric Fault Model

The profile declares:

```
fault_model:
  crash: f          # nodes that can crash (crash-stop)
  byzantine: m      # byzantine nodes (arbitrary responses)
  omission: o       # nodes with message loss
quorum: q           # nodes needed to decide
total_nodes: n      # total nodes in deployment
```

Resilience Bounds

Fault Model	Condition	Quorum	Reference
Crash-stop only	`n ≥ 2f + 1`	`q = floor(n/2) + 1`	Raft, Paxos
Crash + omission	`n ≥ 2(f+o) + 1`	`q = f + o + 1`	Paxos with omission
Byzantine	`n ≥ 3m + 1`	`q = 2m + 1`	PBFT, Tendermint

> **Known issue:** The current contract uses `quorum = f + 1` for crash-stop, which is correct when `n = 2f + 1` but **not as a general formula**. For `n = 5, f = 1`, `f + 1 = 2` but two quorums of 2 nodes may not intersect. The correct general formula is `q = floor(n/2) + 1`. This should be corrected in the next revision.

Fault Model Schema

The catalog defines a strict schema for the fault model:

```
fault_model_schema:
  crash:
    type: "integer"
    minimum: 0
  byzantine:
    type: "integer"
    minimum: 0
  omission:
    type: "integer"
    minimum: 0
  quorum:
    type: "integer"
    minimum: 1
  total_nodes:
    type: "integer"
    minimum: 1
```

The profile verifier should reject any configuration violating these bounds. (Note: the current `validate_profiles.py` does not yet validate `fault_model` parameters — this is a known gap.)

---

6. Evidence and Conformance

REG+ defines 16 evidence types in `evidence-type-specs-v1.md`. Each capability in the catalog maps to required evidence types. The verifier (`verify_effective_guarantees.py`) operates **fail-closed**: a claimed capability without valid evidence (file exists, SHA-256 matches, path is safe) causes the build to fail.

Four Levels of Evidence Strength

Level	What is established	Current status
1. Integrity	File matches declared hash	✅ Implemented
2. Traceability	File linked to commit, command, environment	⚠️ Partially (declarative)

Level	What is established	Current status
3. Reproducible execution	Third party can rerun and compare	□ Not implemented
4. Independent attestation	Authenticated verifier attests results	□ Not implemented

Current evidence is **self-attested** (level 1-2). The hash proves the file is unchanged since attestation, not that the test was executed correctly. Independent attestation requires a third-party executor and cryptographic signature.

Normative Invariants in the Catalog

The capability catalog (`capabilities-v1.yaml` v1.1.0) defines three normative invariants:

ID	Name	Property	Scope
I8	budget-ceiling-aggregate	`reserved + committed ≤ capacity`	per `resource_id`
LIN	linearizability	linearizable history exists	all operations
AGR	agreement	at most one commit per reservation	all commits

These invariants are normative: any conforming backend must maintain them. The verifier checks that evidence exists for each claimed capability, and the evidence must demonstrate the invariant holds.

---

7. Use Case 1: Financial Mandate Enforcement

Scenario: A fund management firm deploys multiple AI agents that propose trades against a shared portfolio. Each agent operates independently but all draw from the same mandate ceiling.

Problem without REG+: Three agents each see $60M available in a $100M mandate. Without aggregate budget coordination, all three could reserve $60M simultaneously, committing $180M against a $100M ceiling.

With REG+:

```
Agent A → reserve("trade-A", 60M, "mandate-EU-equity")
  → RESERVED (reserved=60, committed=0, available=40)

Agent B → reserve("trade-B", 60M, "mandate-EU-equity")
  → INSUFFICIENT_BUDGET (available=40 < 60)

Agent C → reserve("trade-C", 40M, "mandate-EU-equity")
  → RESERVED (reserved=100, committed=0, available=0)
```

Agent A executes the trade → `commit("res-A")` → `reserved=40, committed=60`
Agent A's trade fails → `release("res-A")` → `reserved=0, committed=0` (budget returned)

If Agent A's trade outcome is unknown (network timeout to broker):

```
reconcile("trade-A") → queries broker
  → effect_observed=True → COMMITTED (reserved=0, committed=60)
  → effect_observed=False → FAILED/RELEASED (reserved=0, committed=0)
  → effect_observed=None → UNRESOLVED (stays UNKNOWN, human review)
```

I8 holds throughout: at no point does `reserved + committed > 100`.

Deployment: Single-host with `SingleHostLockBackend` for a single trading desk; multi-host with `RaftConsensusBackend` for globally distributed trading desks sharing the same mandate.

---

8. Use Case 2: Cloud Infrastructure Budget Governance

Scenario: A DevOps team deploys autonomous agents for deployment, optimization, and remediation. All agents consume from a shared cloud budget (API calls, compute minutes, storage).

Problem: Each agent respects its individual rate limit, but the aggregate across 12 agents exceeds the organizational cloud budget envelope. The CFO needs a guarantee that total committed cloud spend never exceeds the approved budget.

With REG+:

```
from tenir_governance.operations.consensus_backend import SingleHostLockBackend

backend = SingleHostLockBackend(store_dir="/var/lib/tenir/budget")
backend.register_resource("cloud-budget-2026", 50000)  # $50K monthly

# Each agent reserves before making API calls
result = backend.reserve("deploy-prod-v123", 500, "cloud-budget-2026")
if result.status == "RESERVED":
    try:
        execute_deployment()
        backend.commit(result.reservation_id)  # committed += 500
    except Exception:
        backend.release(result.reservation_id)  # budget returned
else:
    # INSUFFICIENT_BUDGET — agent must wait or use smaller batch
    pass
```

Reconciliation scenario: An agent makes a provisioning API call but the response is lost (timeout). The reservation is in UNKNOWN state. REG+ reconciliation queries the cloud provider's API to determine whether the resources were actually provisioned:

```
def provider_reconcile(key):
    # Query AWS/GCP API for the operation
    response = cloud_client.get_operation_status(key)
    if response.status == "DONE":
        return True   # effect observed → commit
    elif response.status == "FAILED":
        return False  # no effect → release
    else:
        return None   # indeterminate → stays UNKNOWN

backend = SingleHostLockBackend(
    store_dir="/var/lib/tenir/budget",
```

```
    provider_reconcile=provider_reconcile
)
```

Key value: The CFO gets a verifiable guarantee that cloud spend never exceeds budget, with automatic reconciliation for uncertain operations. No agent can bypass the budget gate.

---

9. Use Case 3: Multi-Agent Procurement Coordination

Scenario: A large organization has multiple departments, each with autonomous procurement agents that can issue purchase orders. All POs draw from a shared quarterly procurement envelope. The CFO needs to prevent aggregate over-commitment while allowing each department to operate independently.

With REG+ (multi-host Raft):

```
from tenir_governance.operations.raft_backend import RaftConsensusBackend

# Deploy 3 Raft nodes across 3 data centers
node_east = RaftConsensusBackend(
    self_addr="dc1.internal:5000",
    other_addrs=["dc2.internal:5000", "dc3.internal:5000"],
    store_dir="/var/lib/tenir/raft",
    provider_reconcile=erp_reconcile_function
)

node_east.register_resource("procurement-Q4-2026", 2_000_000)

# Agent in EU reserves against the shared budget
result = node_east.reserve("po-eu-laptops-2026", 500_000, "procurement-Q4-2026")
# Replicated to all 3 nodes — I8 checked on leader before commit

# Agent in US reserves simultaneously
result = node_west.reserve("po-us-servers-2026", 1_500_000, "procurement-Q4-2026")
# If EU reservation already committed: only 1.5M available
# → RESERVED if available >= 1.5M, INSUFFICIENT_BUDGET otherwise
```

Fault tolerance: If dc1 crashes, dc2 and dc3 maintain quorum (2 of 3). Operations continue. When dc1 recovers, it catches up via the Raft log. No double-commit, no phantom reservations.

Budget conservation across hosts: I8 is enforced on the leader before the Raft command is committed. Even under leader failover, the replicated state machine ensures `reserved + committed ≤ capacity` on all nodes.